TDT4136 INTRODUCTION TO ARTIFICIAL INTELLIGENCE · CHAPTER 24

Natural Language Processing

Massimiliano Ruocco, PhD

Associate Professor, IDI, NTNU

READING

Main reference

Russell & Norvig, Artificial Intelligence: A Modern Approach, 4th edition, 2021

Chapter 24 — Natural Language Processing

The sections in scope for the exam are listed on the last slide.

INTRODUCTION

Natural language and humans

- Language is how we pass on knowledge
- Turing based his test for intelligence on language

Speaker

Has a goal, plans the words

→

Language

The action that carries the meaning

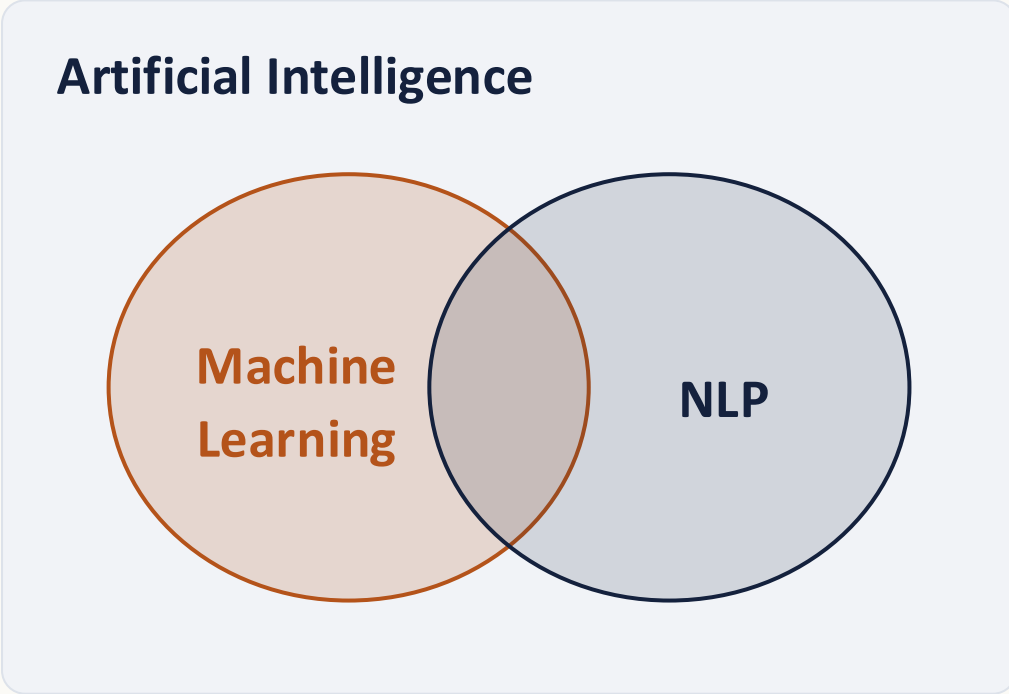
→

Listener

Infers the intended meaning

What is NLP?

Natural Language Processing (NLP) is the area of AI concerned with enabling computers to understand, generate and interact using human language.



INTRODUCTION

Why should computers understand language?

To communicate with humans

Easier than formal logic

To learn from what humans write

Most knowledge is written text (Wikipedia: ~30 million pages)

To understand language scientifically

With linguistics, cognitive psychology and neuroscience

INTRODUCTION

Why language is hard for machines

Ambiguous

“He saw her duck”

“I saw the man with the telescope”

Vague

“That’s great!”

“It’s kind of cold”

Context-dependent

“I’m here today”

“Let’s meet after lunch”

Evolving

New words and senses: “tweet”, “cloud”, “stream”

Judgments vary

“Not to be invited is sad” vs “To be not invited is sad”

No fixed reference

Two uses of “Richard” may refer to different people

PART 1 · SECTION 24.1

Language models

Bag-of-words · n-grams · smoothing · comparing models

KEY CONCEPT

A language model is a probability distribution over strings

P(“Do I dare disturb the universe?”) ≫ P(“Universe dare the I disturb do?”)

- Predicts the next word: completions for e-mails and messages
- Suggests spelling and grammar corrections
- Finds the most probable translation or answer
- Always an approximation: “All grammars leak” (Sapir, 1921)

The bag-of-words model

- Sentence as an unordered collection of words
- Based on the **naïve Bayes** assumption: each word is generated independently, given the text's class or topic

$$P(\text{Class} \mid w_1{:}N) \propto P(\text{Class}) \prod_j P(w_j \mid \text{Class})$$

"Stocks rallied on Monday, with major indexes gaining 1% as optimism persisted…"

→ **Business**

"Heavy rain continued to pound much of the east coast on Monday, with flood warnings…"

→ **Weather**

A generative story: drawing words from a bag

One bag per category; frequent words on more slips

1 Pick a bag (category) and discard the others.

2 Draw a word at random, put it back, draw again.

3 Stop when an end-of-sentence marker (a period) is drawn.

Clearly wrong, but useful

Words are not independent → incoherent text

But good for classification: “stocks” → business, “rain” → weather

Training the bag-of-words model

Estimate by counting in a labelled corpus

3,000 texts, 300 labelled business:

$$P(\text{business}) \approx 300 / 3{,}000 = 0.1$$

100,000 words in business texts, 700 are “stocks”:

$$P(\text{stocks} \mid \text{business}) \approx 0.007$$

- Needs labelled texts
- Rare words → smoothing (24.1.4)
- Tokenization: is “aren’t” one token or two?

Word order matters

Bag-of-words ignores order. The word “quarter” is common in both business and sports texts, but its neighbours tell them apart:

“first **quarter** earnings report”

→ **Business**

“fourth **quarter** touchdown passes”

→ **Sports**

We need a model in which each word depends on the words before it.

From the chain rule to the Markov assumption

Exact: each word depends on all previous words

$$P(w_{1:N}) = \prod_j P(w_j \mid w_{1:j-1})$$

Captures every interaction, but impractical: 100,000-word vocabulary and 40-word sentences → about 10^{200} parameters.

n-gram (Markov) model: only the n − 1 previous words matter

$$P(w_j \mid w_{1:j-1}) \approx P(w_j \mid w_{j-n+1:j-1}) \quad \text{so} \quad P(w_{1:N}) \approx \prod_j P(w_j \mid w_{j-n+1:j-1})$$

chain rule
each word given all previous

→

Markov: keep n − 1
only the last n − 1 words

→

∏ short contexts
few parameters, from counts

Choosing n

n = 1

Unigram

$P(w_j)$

Word frequencies only, like bag-of-words: ignores context.

n = 2

Bigram

$P(w_j \mid w_{j-1})$

e.g. P(cat | black)

Each word depends on one predecessor.

n = 3

Trigram

$P(w_j \mid w_{j-2}, w_{j-1})$

e.g. P(cat | a, black)

Looks back two words.

Bigrams and trigrams capture most useful local structure. Higher n is more realistic, but runs into sparse data.

What n-gram models are good at

Newspaper sections

Classifying articles as business, sports, weather, …

Spam detection

Distinguishing spam e-mail from legitimate mail

Sentiment analysis

Is a movie or product review positive or negative?

Author attribution

Hemingway’s style and vocabulary differ from Faulkner’s or Shakespeare’s

Character-level models

- Each character depends on the previous n − 1
- Handles unseen words: “dextroamphetamine”, “Kallenberger”, “Plattsburg”

$$P(c_j \mid c_{j-n+1:j-1})$$

DANISH COMPOUND WORD

Speciallægepraksisplanlægningsstabiliseringsperiode

Language identification

“Hello, world” → English

“Wie geht’s dir” → German

> 99%

even on short texts · **Swedish vs Norwegian**: ≈ 95%

Skip-gram models

- n-grams miss words that are far apart
- Skip-grams: pairs of nearby words, skipping some in between

FRENCH EXAMPLE

"je ne comprends pas"

1-skip-bigrams: **"je comprends"**, **"ne pas"**

Conjugation (je … comprends) and negation (ne … pas)

The problem: sparse data and zero counts

- Rare n-grams → noisy estimates
- Unseen n-grams → P = 0 for the whole sentence
- **Goal:** keep some probability for the unseen

1 · Unknown words

Words never seen in training

2 · Unseen n-grams

“colorless aquamarine ideas”: known words, new order

Problem 1: unknown words

- Rare words → **<UNK>** (e.g. keep the top 50,000)
- Unknown word at test time → P(<UNK>)

Typed tokens

<NUM> numbers

<EMAIL> e-mail addresses

<URL> web addresses

<S> start / end of text

⚠ Unseen n-grams of known words still get 0 → smoothing

Laplace (add-one) smoothing

Origin: will the sun rise tomorrow?

The sun rose on all N ≈ 2 million days. Zero failures, but P should not be 0.

For language models: add 1 to every count

$$P(w_i \mid w_{i-1}) = (c(w_{i-1}\, w_i) + 1) / (c(w_{i-1}) + V)$$

V = vocabulary size. Too much mass goes to unseen events.

$c + 1$
no count is zero

→

$+ V$ in the denominator
still sums to 1

→

$V = 100{,}000$
too much mass to the unseen

Backoff and linear interpolation

Backoff: one order at a time

Use the trigram if its count is high enough.

Low or zero count → back off to the bigram.

Still unseen → back off to the unigram.

Linear interpolation: always mix all orders

$$\hat{P}(w_i \mid w_{i-2:i-1}) = \lambda_3 P(w_i \mid w_{i-2:i-1}) + \lambda_2 P(w_i \mid w_{i-1}) + \lambda_1 P(w_i)$$

$\lambda_3 + \lambda_2 + \lambda_1 = 1$ · fixed, learned with EM, or count-dependent

λ_3 × trigram
most specific

→

λ_2 × bigram
more data

→

λ_1 × unigram
never zero

→

$= \hat{P}$
smoothed

Sampling from n-gram models of the AI textbook

n	Random sample
1 (unigram)	logical are as are confusion a may right tries agent goal the was
2 (bigram)	systems are very similar computational approach would be represented
3 (trigram)	planning and scheduling are integrated the success of naive Bayes model is
4 (4-gram)	taking advantage of the structure of Bayesian networks and developed various languages for writing “templates” with logical variables, from which large networks could be constructed automatically for each problem instance

The limits of n-gram models

- Larger n → more fluent, but copies the training text
- Better models need structure or learned word representations

Looking ahead: transformers

GPT-2 (Chapter 25): fluent and on topic

But the sentences don’t build a coherent argument

PART 2 · SECTIONS 24.1.5 – 24.1.6

Word representations and part-of-speech tagging

WordNet · Penn Treebank · HMM tagging · logistic regression · generative vs discriminative

n-gram models learn from counts only

“a black cat”

Seen in about 0.000014% of training trigrams → valid

“cat black a”

Never seen → invalid

- n-gram models are **atomic**: each word is a distinct symbol with no internal structure or relations to other words.

Humans generalize beyond the data

"the fulvous kitten" is recognisably English, even if we have never heard it:

Morphology

Words ending in "-ous" are almost always adjectives

Syntax

"a" and "the" play the same role: article + adjective + noun

Semantics

"kitten" is closely related to "cat"

So seeing "a black cat" is evidence that "the fulvous kitten" is valid too.

We need structured word models

Problem: words as atoms

Words are indivisible IDs, so no:

- morphology (-ous, -ed, -ing)
- syntactic roles (noun, verb, adjective)
- semantic classes (cat, kitten, dog)

Solution: factored or structured models

Describe words by properties and relations:

- Hand-built dictionaries, e.g. WordNet (next)
- Part-of-speech tags (this part)
- Learned word embeddings (Section 25.1)

WordNet: a hand-built dictionary

- Open-source, machine-readable lexical database, built by hand.
- Records relations: IS-A (hypernym), PART-OF, antonym, …
- Limit: gives categories, not details. It won’t tell you what a kitten looks like or how it behaves.

Entry for “kitten”

noun.animal: young domestic cat

kitten → young_mammal → mammal → animal

verb.body: give birth to kittens

“our cat kittened again this year”

Parts of speech

- Part of speech (tag): noun, verb, adjective, …
- Adjectives before nouns in English, after them in French

POS tagging

Give each word its part of speech

First step for QA, translation, text-to-speech (“record”)

The Penn Treebank tag set

45 tags, used to annotate a corpus of over 3 million words. A tagged sentence from the corpus:

From	the	start	,	it	took	a	person	with	great	qualities	to	succeed
IN	DT	NN	,	PRP	VBD	DT	NN	IN	JJ	NNS	TO	VB

IN preposition · **DT** determiner · **NN** noun, singular · **NNS** noun, plural · **PRP** personal pronoun · **VBD** verb, past tense · **JJ** adjective · **TO** “to” · **VB** verb, base form

The right tag depends on context

Word	Possible tags	Examples
can	NN (noun), MD (modal)	a tin **can** (NN) · I **can** swim (MD)
flies	NNS (noun), VBZ (verb)	fruit **flies** (NNS) · time **flies** (VBZ)
record	NN (noun), VB (verb)	play the **record** (NN) · **record** the song (VB)
cut	NN, VBD, VBP	Yesterday I **cut** the rope (VBD) · Now I **cut** the rope (VBP)

So a tagger must choose the whole sequence of tags that best fits the sentence, not each tag in isolation.

A hidden Markov model for tagging

Evidence: the words W_1:N

Hidden states: the tags C_1:N

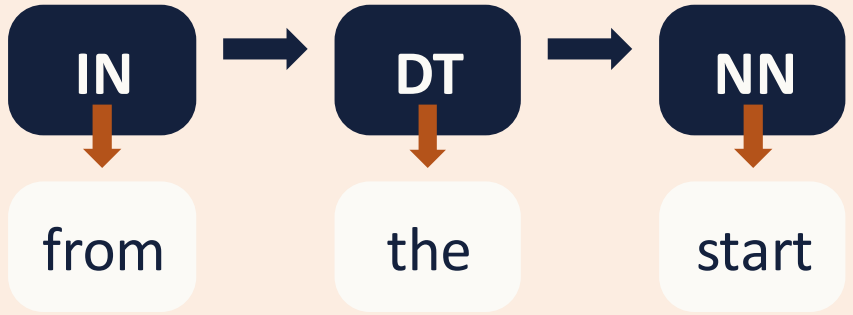
TRANSITION MODEL

$$P(C_t \mid C_{t-1})$$

SENSOR (EMISSION) MODEL

$$P(W_t \mid C_t)$$

Generative story: in each state, emit a word, then choose the next state.



Dark arrows: transitions between tags · orange arrows: emitted words

$$P(C_1{:}N, W_1{:}N) = P(C_1) \prod_{t=2}^{N} P(C_t \mid C_{t-1}) \prod_{t=1}^{N} P(W_t \mid C_t)$$

$P(C_1)$	$\times \prod P(C_t \mid C_{t-1})$	$\times \prod P(W_t \mid C_t)$
first tag	tag → next tag	tag → word

Training the HMM tagger: counting

TRANSITION

$$P(C_t = VB \mid C_{t-1} = MD) = 0.8$$

After a modal verb (would), a base-form verb (think) follows 80% of the time.

Penn Treebank: 13,124 MD tokens, 10,471 followed by VB → 10,471 / 13,124 ≈ 0.80

SENSOR

$$P(W_t = \text{would} \mid C_t = MD) = 0.1$$

When the model chooses a modal verb, it emits would 10% of the time.

Both models are estimated from corpus counts with smoothing, just as for n-grams.

Decoding with the Viterbi algorithm

- Goal: the most probable tag sequence for the whole sentence.
- Viterbi (Section 14.2.3) is dynamic programming: for each word and each tag, keep only the best path that ends there.

$$\hat{C}_{1:N} = \text{argmax}\ P(C_{1:N}, W_{1:N})$$

the argmax is over all tag sequences $C_{1:N}$

word × tag grid
one cell per word and tag

➡

best path per cell
transition × emission

➡

back-pointers
→ $\hat{C}_{1:N}$

≈ 97%

tagging accuracy, from a model that knows nothing about meaning

Limitations of the HMM tagger

- A tag depends only on the current word and the previous tag, not on wider context.
- There is no easy way to add explicit linguistic knowledge, such as:

"Words ending in **-ous** are likely adjectives"

"In **attorney general**, attorney is a noun, not an adjective"

Logistic regression can represent this kind of information.

Logistic regression tagging

- Discriminative: learns P(C | W)
- One classifier per tag (45), trained by gradient descent

$$P(\text{tag} \mid x) = \sigma(w \cdot x)$$



Typical features

Nearby words · suffixes, prefixes · previous tag · capitals, digits · tense of nearby words

Decoding: from single words to a sequence

One word at a time → we need a search

Greedy search

$c_i = \text{argmax}\ P(c' \mid w_{1:N}, c_{1:i-1})$

Left to right, best tag each step. Fast, but can’t go back.

Viterbi

Best path to every tag, so it can change its mind. Slower.

Beam search

Keep only the b best. b trades speed for accuracy.

Generative vs discriminative models

Aspect	Generative (naive Bayes, HMM)	Discriminative (logistic regression)
Learns	Joint P(W, C)	Conditional P(C | W)
Can generate sentences?	Yes, by sampling	No
Features	Limited to the model's structure	Arbitrary hand-crafted features
Error rate	Generally higher	Generally lower
Training	Converges faster; preferred with little data or training time	Converges more slowly

Both can tag well: the HMM tagger already reaches about 97% accuracy.

PART 3 · SECTIONS 24.2 – 24.3

Grammar and parsing

PCFGs · the E_0 grammar · chart parsing · CYK · faster parsers

From words to structure

Grammar

A set of rules that defines the tree structure of allowable phrases

Language

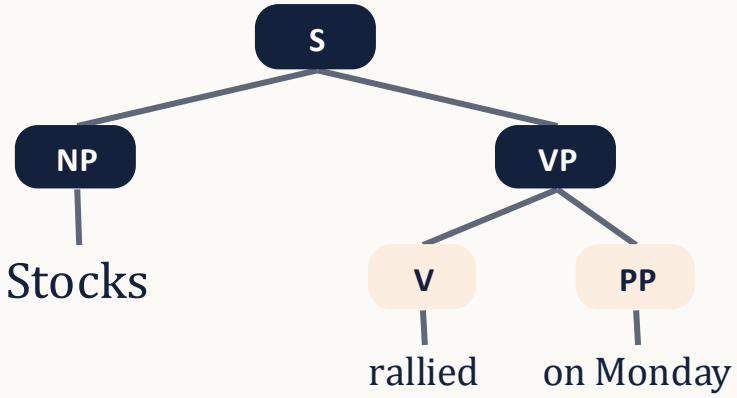
The set of sentences that follow those rules

Feature	Formal language (first-order logic, BNF)	Natural language
Boundary	Clear: valid vs invalid sentences	Fuzzy, flexible usage
Tree structure	Unique, well defined	Often several possible parses

Hierarchical structure in natural language

"Stocks rallied on Monday."

Phrase	Category	Role
Stocks	Noun phrase (NP)	Subject
rallied on Monday	Verb phrase (VP)	Predicate
on Monday	Prepositional phrase (PP)	Modifier of the VP



Syntactic categories (NP, VP, PP) constrain which words are probable at each point, and the phrase structure gives a framework for the meaning (semantics) of the sentence.

Probabilistic context-free grammars (PCFG)

Rules say how phrases combine, probabilities how likely

- **Probabilistic:** every rule has a probability
- **Context-free:** a rule works in any context

Example rule

```
Adjs → Adjective [0.80]
     | Adjective Adjs
[0.20]
```

One adjective 80%, several (“smelly dead”) 20%

The E_0 grammar for the wumpus world

- **Purpose:** a tiny English-like PCFG for communication between agents exploring the wumpus world.
- **Syntactic categories:** sentence (S), noun phrase (NP), verb phrase (VP), list of adjectives (Adjs), prepositional phrase (PP), relative clause (RelClause).

Sentences E_0 generates

"I feel a breeze."

"It stinks."

"The smelly dead wumpus is in 1 3."

The rules of E_0

S	→ NP VP	0.90	I + feel a breeze
	| S Conj S	0.10	… + and + It stinks
NP	→ Pronoun	0.25	I
	| Name	0.10	Ali
	| Noun	0.10	pits
	| Article Noun	0.25	the + wumpus
	| Article Adjs Noun	0.05	the + smelly dead + wumpus
	| Digit Digit	0.05	3 4
	| NP PP	0.10	the wumpus + in 1 3
	| NP RelClause	0.05	the wumpus + that is smelly
	| NP Conj NP	0.05	the wumpus + and + I

VP	→ Verb	0.40	stinks
	| VP NP	0.35	feel + a breeze
	| VP Adjective	0.05	smells + dead
	| VP PP	0.10	is + in 1 3
	| VP Adverb	0.10	go + ahead
Adjs	→ Adjective	0.80	smelly
	| Adjective Adjs	0.20	smelly + dead
PP	→ Prep NP	1.00	to + the east
RelCl.	→ RelPro VP	1.00	that + is smelly

Limits of the E_0 grammar

Overgeneration

Generates sentences that are not grammatical

"Me go I."

Undergeneration

Rejects sentences that are valid English

"I think the wumpus is smelly."

- Better coverage comes from learning grammars from treebanks (Section 24.3.2, outside this course's scope).

Parsing as search

Parsing = search for a parse tree whose leaves are the words

Top-down

From S down to the words

Bottom-up

From the words up to S

Both can repeat work or hit dead ends

Parsing “the wumpus is dead” with E_0

List of items	Rule applied
S	
NP VP	S → NP VP
NP VP Adjective	VP → VP Adjective
NP Verb Adjective	VP → Verb
NP Verb dead	Adjective → dead
NP is dead	Verb → is
Article Noun is dead	NP → Article Noun
Article wumpus is dead	Noun → wumpus
the wumpus is dead	Article → the

Read downwards: top-down

Replace a nonterminal X by the right-hand side of a rule X → Y …

Read upwards: bottom-up

Replace a matching sequence Y … by X, e.g. “the” → Article, then Article Noun → NP.

Avoiding repeated work: chart parsing

Have the students in section 2 of Computer Science 101 **take** the exam. (command)
Have the students in section 2 of Computer Science 101 **taken** the exam? (question)

Same first 10 words. Wrong guess at word 1 → found at word 11 → redo everything.

- Store each phrase in a **chart**: “the students in … 101” = NP
- Context-free → reuse it in every branch

The CYK algorithm

- Bottom-up, probabilistic chart parser
- Best parse for the sentence and for every part of it

P[X, i, k]
best X over words i…k

➡

max over split j
Y over i…j, Z over j+1…k

➡

P[Y] × P[Z] × p
rule X → Y Z [p]

Requires Chomsky Normal Form

```
X → word [p]
X → Y Z [p]
```

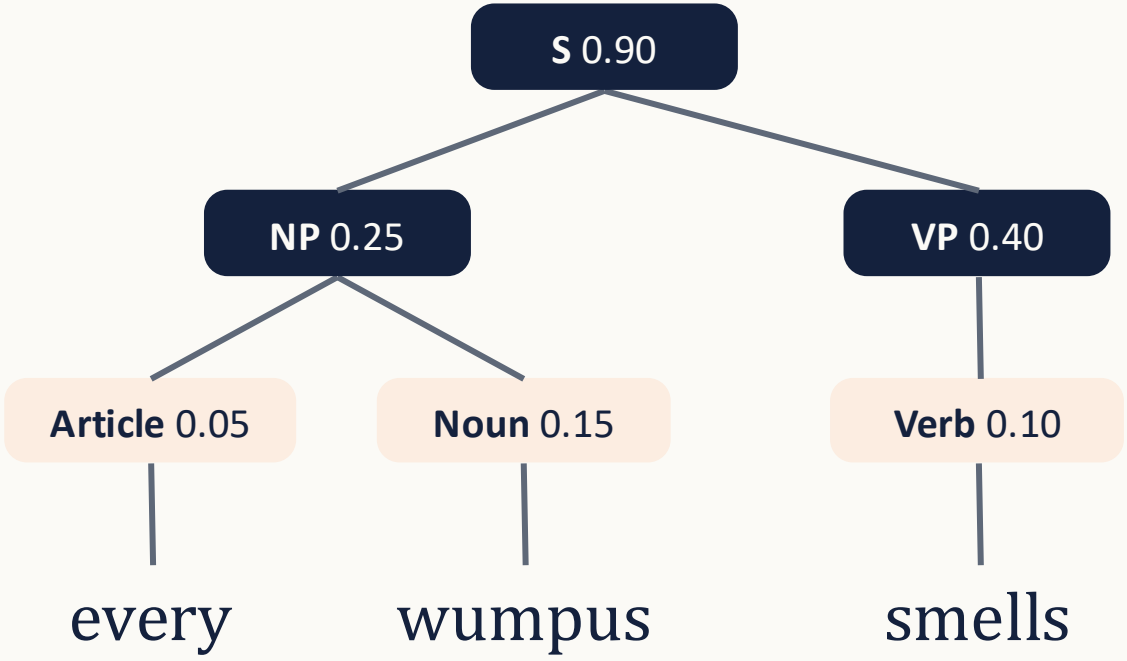
Any context-free grammar can be converted automatically.

Complexity

Time $O(n^3 m)$, space $O(n^2 m)$

n = words, m = nonterminals

The probability of a parse tree



Each node carries the probability of the rule used there. The tree's probability is their product:

$$0.90 \times 0.25 \times 0.05 \times 0.15 \times 0.40 \times 0.10$$

$$= 0.0000675$$

Linear form: [S [NP [Article every] [Noun wumpus]] [VP [Verb smells]]]

Faster parsing: towards O(n)

Natural language should allow faster parsing than general CFGs

A* search

Start = words, goal = S, cost from rule probabilities

Admissible heuristic → first parse is the most probable

Beam search

Keep the b best partial parses

O(n), usually finds the best parse

Shift-reduce

Beam with b = 1

Shift a word onto a stack, or reduce with a rule

PART 4 · SECTION 24.6

Natural language tasks

Speech · translation · extraction · retrieval · question answering

NLP bridges speech, text and meaning

Speech recognition

Sound → text

Text-to-speech

Text → sound

Machine translation

One language → another

Information extraction

Text → structured facts

Information retrieval

Query → relevant documents

Question answering

Question → answer

Speech recognition

- Sound → text
- Word error ≈ 3–5%, like human transcribers
- In the book: RNNs combined with HMMs

≈ 30%

drop in error rate with deep neural networks (2011)

Text-to-speech synthesis

The inverse of speech recognition: from text to spoken sound.

- Pronounce each word correctly (“record”)
- Natural pauses and emphasis
- Different voices

$\approx 2/3$

of listeners preferred the neural WaveNet (2016)

Machine translation

Trained on paired documents (e.g. English–French), no annotation needed

2000s

n-gram models

Meaning OK, syntax errors; n-grams of at most 7 words

2014

Recurrent seq2seq

Word embeddings, whole-sentence context

2017

Transformers

Attention: big quality boost

Hybrids

Near human level

Near-human on some language pairs

Information extraction

Goal

Find objects and their relations in text

Addresses → (street, city, state, zip)

Storms → (temperature, wind, precipitation)

Methods

Tables: regular expressions

Open relations: TEXTRUNNER

Free text: HMMs, rule learning (NELL)

Today: neural networks with embeddings

Information retrieval

Find the documents that are relevant and important for a given query.

- The core of web search engines such as Google and Baidu, run billions of times a day.
- Results are ranked by relevance and importance.

Question answering

Returns an answer, not a list of documents

Period	Approach
1960s →	Systems based on syntactic parsing
1997	START parser and question answerer (Katz)
2001 →	Web retrieval radically increases coverage, e.g. ASKMSR (Banko et al., 2002)
Evaluation	TREC competitions (since 1991), AI2 ARC science questions (Clark et al., 2018)

How ASKMSR answers

"Who founded the U.S. Coast Guard?"

Search patterns:

[* founded the U.S. Coast Guard]

[the U.S. Coast Guard was founded by *]

"Who" → a person → **Alexander Hamilton**

WRAP-UP

Summary

Level	What we covered	Modern extension
Language models	Bag-of-words / naive Bayes; n-grams (Markov assumption); <UNK>; smoothing (Laplace, backoff, interpolation)	Neural language models
Words and tags	WordNet; POS tagging with HMM + Viterbi and logistic regression; generative vs discriminative	Word embeddings, neural taggers
Structure	Grammars, PCFGs, the E_0 grammar	Neural syntactic models
Parsing	Top-down / bottom-up, chart parsing, CYK; A*, beam search, shift-reduce	Learned neural parsers
Applications	Speech recognition, TTS, translation, IE, IR, QA	Seq2seq and transformer models

Language models → word representations → grammar → parsing → applications

WRAP-UP

Reading and exam scope

Reference

Russell & Norvig, Artificial Intelligence: A Modern Approach, 4th edition, 2021, Chapter 24

All sections are in scope, except those listed on the right.

Excluded

- 24.2.1 The lexicon of E_0
- 24.3.1 Dependency parsing
- 24.3.2 Learning a parser from examples
- 24.4 Augmented grammars
- 24.5 Complications of real natural language